

FUNDAMENTALS OF PROGRAMMING

MOCK FINAL EXAM

Faculty of Information Technology
VNUHCM, University of Science

Time Allotted: 120 Minutes

Instructions:

1. There are **5 problems** in this contest.
2. Solve as many problems as possible within the given time.
3. You must strictly follow memory management guidelines (avoid memory leaks for dynamic allocation problems).
4. Restricted Language: **C++** (utilize `<iostream>`, `<fstream>`, `<cstring>`, pointers, and structs as taught).

Problem 1: Number Investigator**Time Limit:** 1.0 seconds **Memory Limit:** 256 MB**Input File:** standard input **Output File:** standard output

As a warm-up, you need to analyze a series of integers. Using your knowledge of branching and loops, write a program that analyzes an integer n and checks three properties:

1. **Symmetric:** Is n a palindrome? (e.g., 121 is symmetric, 123 is not).
2. **Square:** Is n a perfect square? (e.g., 4, 9, 16 are squares).
3. **Prime:** Is n a prime number? (e.g., 2, 3, 5 are primes).

For each test case, output "Yes" or "No" for each property.

Input

- The first line contains an integer T ($1 \leq T \leq 100$), the number of test cases.
- Each of the next T lines contains a single positive integer n ($1 \leq n \leq 10^9$).

Output

- For each test case, print three words separated by spaces: "Yes" or "No" corresponding to Symmetric, Square, and Prime properties, respectively.

Sample Input	Sample Output
3 121 4 13	Yes Yes No Yes Yes No No No Yes

Problem 2: The Text Normalizer**Time Limit:** 1.0 seconds **Memory Limit:** 256 MB**Input File:** standard input **Output File:** standard output

You are given a messy string containing words, multiple spaces, and inconsistent capitalization. Your task is to normalize this string using **C-strings (char arrays) and pointers**.

The normalization rules are:

1. Remove all leading and trailing spaces.
2. Multiple spaces between words should be reduced to a single space.
3. Capitalize the first letter of every word.
4. Convert all other letters to lowercase.

Note: You are encouraged to use `<cstring>` functions and pointer arithmetic (e.g., `*(p+i)`) as discussed in the "Advanced Pointer" lectures.

Input

- A single line of text containing English letters and spaces. The length of the string does not exceed 1000 characters.

Output

- Print the normalized string.

Sample Input	Sample Output
heLLo WORLD thiS is C++	Hello World This Is C++

Problem 3: Student Ranking System**Time Limit:** 1.0 seconds **Memory Limit:** 256 MB**Input File:** standard input **Output File:** standard output

You are tasked with managing a student database. Define a struct **Student** with the following fields:

- **id** (integer)
- **name** (string, no spaces for simplicity)
- **score** (float)

Write a program that simulates reading from a file. You will accept N student records. You need to filter out students who have a score less than a threshold K , and then sort the remaining students in **descending order of score**. If two students have the same score, sort them by **ID in ascending order**.

Finally, display the top M students.

Input

- First line: N ($1 \leq N \leq 1000$), K ($0 \leq K \leq 10$), M ($1 \leq M \leq N$).
- Next N lines: Each contains **id name score**.

Output

- Print the top M students (or fewer if fewer than M passed) in the format: **ID Name Score**.
- The score should be formatted to 2 decimal places (optional but recommended).

Sample Input	Sample Output
5 5.0 3 101 Alice 8.5 102 Bob 4.0 103 Charlie 9.0 104 Dave 8.5 105 Eve 9.5	105 Eve 9.5 103 Charlie 9.0 101 Alice 8.5

Problem 4: Dynamic Matrix Transpose**Time Limit:** 2.0 seconds **Memory Limit:** 256 MB**Input File:** standard input **Output File:** standard output

In the "Advanced Pointer" module, you learned about pointers to pointers (`int**`) and dynamic memory allocation.

Your task is to write a function that:

1. Dynamically allocates a 2D array of size $R \times C$.
2. Reads the matrix elements.
3. Computes the **Transpose** of the matrix (a new matrix of size $C \times R$ where element $[i][j]$ becomes $[j][i]$).
4. Prints the transposed matrix.
5. **Crucial:** Properly deallocate (free) all memory used to prevent memory leaks.

Input

- First line: two integers R and C ($1 \leq R, C \leq 100$).
- Next R lines: each contains C integers representing the matrix.

Output

- Print the transposed matrix (C rows and R columns).

Sample Input	Sample Output
2 3 1 2 3 4 5 6	1 4 2 5 3 6

Problem 5: The Train Carriage Manager**Time Limit:** 2.0 seconds **Memory Limit:** 256 MB**Input File:** standard input **Output File:** standard output

You are managing a train using a **Singly Linked List**. Each node represents a carriage with a specific ID. You need to support the following operations efficiently:

- **ADD_HEAD** *X*: Add a carriage with ID *X* to the front (locomotive).
- **ADD_TAIL** *X*: Add a carriage with ID *X* to the end.
- **DEL** *X*: Delete the **first occurrence** of carriage ID *X*. If not found, do nothing.
- **REVERSE**: Reverse the entire train order (Concept from *Linked List Advanced Operations*).

After performing all operations, print the final configuration of the train from head to tail.

Input

- First line: *Q* ($1 \leq Q \leq 1000$), the number of operations.
- Next *Q* lines: Each contains a command as described above.

Output

- Print the IDs of the carriages in the linked list separated by " -> ".
- If the train is empty, print "EMPTY".
- Print "NULL" at the end of the list.

Sample Input	Sample Output
5 ADD_HEAD 10 ADD_TAIL 20 ADD_HEAD 5 REVERSE DEL 10	20 -> 5 -> NULL